

ANALYTICAL PROBLEM SOLVING

Warehouse Delivery Robot Pathfinding

Problem Statement

A delivery robot operates in a warehouse modelled as a 5x5 grid. It starts at cell (1,1) and must deliver a package to cell (5,5). The cells (2,2), (2,3), (3,3), (4,2) and (4,4) are blocked by shelving units. The robot moves Up, Down, Left or Right (no diagonals) and every move costs 1.

Notation: cells are written (row, column); row 1 is the top row and column 1 is the leftmost column.

Warehouse grid (S = start, G = goal, X = shelf)

r1	(1,1) S	(1,2)	(1,3)	(1,4)	(1,5)
r2	(2,1)	(2,2) X	(2,3) X	(2,4)	(2,5)
r3	(3,1)	(3,2)	(3,3) X	(3,4)	(3,5)
r4	(4,1)	(4,2) X	(4,3)	(4,4) X	(4,5)
r5	(5,1)	(5,2)	(5,3)	(5,4)	(5,5) G

Figure 1: Warehouse grid (S = start, G = goal, X = shelf)

Key Observations

- The state space has $25 - 5 = 20$ free cells.
- Cells (3,2) and (4,3) are dead ends: (3,2) is enclosed by (2,2), (4,2) and (3,3); (4,3) is enclosed by (3,3), (4,2) and (4,4).
- The Manhattan distance from (1,1) to (5,5) is $|5-1| + |5-1| = 8$, so no path can cost less than 8.

Part (a) - PEAS Description

a.1 Agent and Task

The agent is a warehouse delivery robot. Its task is to carry a package from (1,1) to (5,5) through the shelving layout shown in Figure 1.

a.2 PEAS Table

Component	Description for the warehouse robot
Performance measure	Reach (5,5) and deliver the package (success); minimise total path cost (number of moves, best = 8); never enter a blocked cell or collide with shelving; minimise time and battery use; deliver the package undamaged.
Environment	5x5 warehouse grid with 20 free cells and 5 blocked shelving cells (2,2), (2,3), (3,3), (4,2), (4,4); start (1,1); goal (5,5); the package carried by the robot.
Actuators	Drive motors / wheels for Up, Down, Left, Right moves; steering and braking; gripper or carrier platform to hold and release the package.
Sensors	Position sensor (wheel encoders, RFID floor tags) giving the current cell; obstacle sensors (ultrasonic / LiDAR / bumper) to detect shelving; camera or marker reader to recognise the goal cell; package-load sensor; battery sensor.

Agent-environment diagram

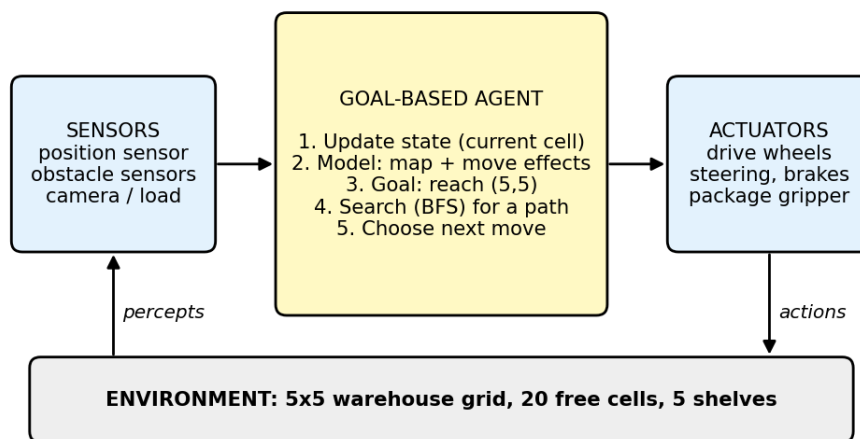


Figure 2: Agent-environment diagram of the goal-based delivery robot

a.3 Environment Properties

Property	Value	Reason
Observability	Fully observable	The grid layout and the robot's own cell are known.
Agents	Single agent	No other robots are modelled.
Determinism	Deterministic	Each move leads to exactly one known next cell.
Episodic / Sequential	Sequential	The current move affects later moves.
Static / Dynamic	Static	Shelves do not move while the robot plans.
Discrete / Continuous	Discrete	Finite set of grid cells and four actions.

Part (b) - Agent Type Classification

b.1 Chosen Agent Type

Goal-based agent.

b.2 Justification Specific to this Scenario

1. The task is defined by a goal. Success means reaching (5,5), not reacting to a single percept. The agent must ask "which sequence of moves gets me to (5,5)?", which is what a goal-based agent does.
2. A simple reflex agent fails. A rule such as "if free, move Right or Down" can walk into the dead ends (3,2) or (4,3) and get stuck or loop, because it has no memory and no look-ahead.
3. Planning is needed because of the shelves. The blocked cells (2,2), (2,3), (3,3), (4,2), (4,4) create dead ends, so the robot must search ahead and choose a path that really reaches the goal.
4. A model is available. The map and the effect of each move are known and deterministic, so the agent can simulate moves internally (an internal model plus a goal).
5. A utility-based agent is unnecessary. Every move costs 1 and there is no trade-off such as risk versus speed; the only requirement is the shortest path, which uninformed search already provides.

b.3 Comparison of Agent Types

Agent type	Suitable?	Reason
Simple reflex	No	No memory or planning; gets trapped in dead ends.
Model-based reflex	Partially	Tracks the state but has no explicit goal to plan towards.
Goal-based	Yes (best)	Uses the map model and a goal test to search for a path.
Utility-based	Not needed	No competing objectives; uniform step cost.

Part (c) - Formal Search Problem Formulation

c.1 State Space

A state is the robot's cell (r, c) with $1 \leq r, c \leq 5$ and (r, c) not in the blocked set $B = \{(2,2), (2,3), (3,3), (4,2), (4,4)\}$. There are 20 valid states.

c.2 Formal Components

Component	Definition
Initial state	$s_0 = (1,1)$
Goal state / goal test	Goal(s) is true if and only if $s = (5,5)$
Actions	ACTIONS(r,c) is a subset of {Up, Down, Left, Right}. An action is applicable only if the resulting cell lies inside the 5x5 grid and is not in B.
Transition model	RESULT((r,c) , Up) = $(r-1, c)$; RESULT((r,c) , Down) = $(r+1, c)$; RESULT((r,c) , Left) = $(r, c-1)$; RESULT((r,c) , Right) = $(r, c+1)$. Moves into the boundary or into B are not allowed.
Path cost	$g(\text{path})$ = sum of step costs; step cost $c(s,a,s') = 1$ for every move, so g = number of moves.

c.3 Examples of the Transition Model

- ACTIONS(1,1) = {Down, Right} (Up and Left leave the grid); RESULT((1,1), Down) = (2,1); RESULT((1,1), Right) = (1,2).
- ACTIONS(3,2) = {Left} only, because Up (2,2), Down (4,2) and Right (3,3) are blocked: a dead end.
- ACTIONS(5,3) = {Up, Left, Right}: Up leads to (4,3), Left to (5,2), Right to (5,4).

c.4 Python Representation

```
from collections import deque
```

```
GRID_SIZE = 5
```

```
BLOCKED = {  
    (2, 2), (2, 3),  
    (3, 3),  
    (4, 2), (4, 4)  
}
```

```
START, GOAL = (1, 1), (5, 5)
```

```
MOVES = [  
    ("Up", -1, 0),  
    ("Down", 1, 0),  
    ("Left", 0, -1),  
    ("Right", 0, 1)
```

```
]
```

```
def is_goal(state):
```

```
    return state == GOAL
```

```
def successors(state):
```

```
    r, c = state
```

```
    result = []
```

```
    for name, dr, dc in MOVES:
```

```
        nr, nc = r + dr, c + dc
```

```
        if (1 <= nr <= GRID_SIZE and  
            1 <= nc <= GRID_SIZE and  
            (nr, nc) not in BLOCKED):
```

```
            result.append((name, (nr, nc)))
```

```
    return result
```

```
def bfs(start, goal):
```

```
    frontier = deque([start])
```

```
    parent = {start: None}
```

```
    action_taken = {start: None}
```

```
    expanded = 0
```

```
    while frontier:
```

```
        state = frontier.popleft()
```

```
        expanded += 1
```

```
        print(f'Step {expanded}: expand {state}')
```

```
        if is_goal(state):
```

```
            path = []
```

```
            actions = []
```

```
            current = state
```

```
            while current is not None:
```

```
                path.append(current)
```

```
                if action_taken[current] is not None:
```

```
                    actions.append(action_taken[current])
```

```
                current = parent[current]
```

```
            path.reverse()
```

```
            actions.reverse()
```

```
            return path, actions, expanded
```

```
    for action, child in successors(state):
        if child not in parent:
            parent[child] = state
            action_taken[child] = action
            frontier.append(child)

    print("Queue:", list(frontier))

    return None, None, expanded

path, actions, nodes_expanded = bfs(START, GOAL)

print("\n===== BFS RESULT =====")

if path:
    print("Path:", path)
    print("Actions:", actions)
    print("Cost:", len(path) - 1)
    print("Nodes Expanded:", nodes_expanded)
else:
    print("No path found.")
```

Output:

```
IDLE Shell 3.13.11
File Edit Shell Debug Options Window Help
Python 3.13.11 (tags/v3.13.11:6278944, Dec 5 2025, 16:26:58) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Nadesa Krishnan/OneDrive/Documents/BFS.py =====
Step 1: expand (1, 1)
Queue: [(2, 1), (1, 2)]
Step 2: expand (2, 1)
Queue: [(1, 2), (3, 1)]
Step 3: expand (1, 2)
Queue: [(3, 1), (1, 3)]
Step 4: expand (3, 1)
Queue: [(1, 3), (4, 1), (3, 2)]
Step 5: expand (1, 3)
Queue: [(4, 1), (3, 2), (1, 4)]
Step 6: expand (4, 1)
Queue: [(3, 2), (1, 4), (5, 1)]
Step 7: expand (3, 2)
Queue: [(1, 4), (5, 1)]
Step 8: expand (1, 4)
Queue: [(5, 1), (2, 4), (1, 5)]
Step 9: expand (5, 1)
Queue: [(2, 4), (1, 5), (5, 2)]
Step 10: expand (2, 4)
Queue: [(1, 5), (5, 2), (3, 4), (2, 5)]
Step 11: expand (1, 5)
Queue: [(5, 2), (3, 4), (2, 5)]
Step 12: expand (5, 2)
Queue: [(3, 4), (2, 5), (5, 3)]
Step 13: expand (3, 4)
Queue: [(2, 5), (5, 3), (3, 5)]
Step 14: expand (2, 5)
Queue: [(5, 3), (3, 5)]
Step 15: expand (5, 3)
Queue: [(3, 5), (4, 3), (5, 4)]
Step 16: expand (3, 5)
Queue: [(4, 3), (5, 4), (4, 5)]
Step 17: expand (4, 3)
Queue: [(5, 4), (4, 5)]
Step 18: expand (5, 4)
Queue: [(4, 5), (5, 5)]
Step 19: expand (4, 5)
Queue: [(5, 5)]
Step 20: expand (5, 5)

===== BFS RESULT =====
Path: [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
Actions: ['Down', 'Down', 'Down', 'Down', 'Right', 'Right', 'Right', 'Right']
Cost: 8
Nodes Expanded: 20
>>>
```

Part (d) - Breadth-First Search

d.1 Algorithm and Assumptions

BFS uses a **FIFO queue** as the frontier and expands the shallowest node first. A visited set avoids repeated states. Children are generated in the order **Up, Down, Left, Right**. The goal test is applied when a node is **expanded** (removed from the queue).

d.2 Pseudocode

```
BFS(problem):
    frontier <- queue containing START
    visited <- {START}
    while frontier is not empty:
        node <- frontier.dequeue()
        if Goal(node): return path(node)
        for each (action, child) in successors(node):
            if child not in visited:
                visited.add(child); parent[child] <- node
                frontier.enqueue(child)
    return failure
```

d.3 Flowchart

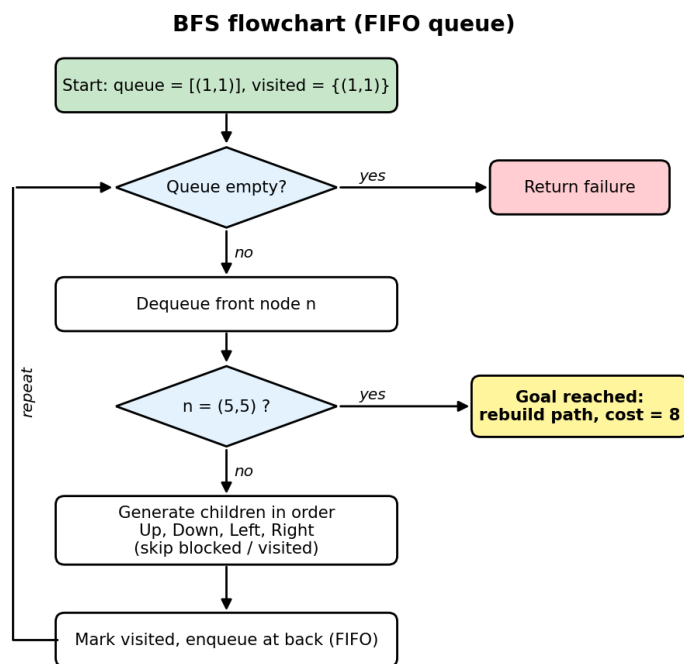


Figure 3: BFS flowchart

d.4 Python Code

```
from collections import deque
start = (1, 1)
goal = (5, 5)
blocked = {
    (2, 2), (2, 3),
    (3, 3),
    (4, 2), (4, 4)
}
moves = [
```

```

(-1, 0), # Up
(1, 0), # Down
(0, -1), # Left
(0, 1) # Right
]
queue = deque([start])
visited = {start}
parent = {start: None}

while queue:
    current = queue.popleft()

    if current == goal:
        break

    r, c = current

    for dr, dc in moves:
        next_cell = (r + dr, c + dc)

        if (1 <= next_cell[0] <= 5 and
            1 <= next_cell[1] <= 5 and
            next_cell not in blocked and
            next_cell not in visited):

            visited.add(next_cell)
            parent[next_cell] = current
            queue.append(next_cell)

path = []
current = goal
while current is not None:
    path.append(current)
    current = parent[current]

path.reverse()

print("BFS Path:")
print(path)
print("Total Cost:", len(path) - 1)

```

Output :

```
Python 3.13.11 (tags/v3.13.11:6278944, Dec 5 2025, 16:26:58) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Nadesa Krishnan/OneDrive/Documents/BFS.py =====
BFS Path:
[(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
Total Cost: 8
>>>
```

d.5 Step-by-Step Expansion (Queue at Each Step)

The queue is written from front to back.

Step	Node expanded	New nodes added	Queue after the step
1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(3,1)	(1,2), (3,1)
3	(1,2)	(1,3)	(3,1), (1,3)
4	(3,1)	(4,1), (3,2)	(1,3), (4,1), (3,2)
5	(1,3)	(1,4)	(4,1), (3,2), (1,4)
6	(4,1)	(5,1)	(3,2), (1,4), (5,1)
7	(3,2)	none (dead end)	(1,4), (5,1)
8	(1,4)	(2,4), (1,5)	(5,1), (2,4), (1,5)
9	(5,1)	(5,2)	(2,4), (1,5), (5,2)
10	(2,4)	(3,4), (2,5)	(1,5), (5,2), (3,4), (2,5)
11	(1,5)	none	(5,2), (3,4), (2,5)
12	(5,2)	(5,3)	(3,4), (2,5), (5,3)
13	(3,4)	(3,5)	(2,5), (5,3), (3,5)
14	(2,5)	none	(5,3), (3,5)
15	(5,3)	(4,3), (5,4)	(3,5), (4,3), (5,4)
16	(3,5)	(4,5)	(4,3), (5,4), (4,5)
17	(4,3)	none (dead end)	(5,4), (4,5)
18	(5,4)	(5,5)	(4,5), (5,5)
19	(4,5)	none	(5,5)
20	(5,5)	GOAL REACHED	empty

d.6 Expansion by Depth Level

Depth	Nodes at this depth
0	(1,1)
1	(2,1), (1,2)
2	(3,1), (1,3)
3	(4,1), (3,2), (1,4)
4	(5,1), (2,4), (1,5)
5	(5,2), (3,4), (2,5)
6	(5,3), (3,5)
7	(4,3), (5,4), (4,5)
8	(5,5)

BFS expansion order (20 nodes)					
	c1	c2	c3	c5	
r1	(1,1) 1	(1,2) 3	(1,3) 5	(1,4) 8	(1,5) 11
r2	(2,1) 2	(2,2) X	(2,3) X	(2,4) 10	(2,5) 14
r3	(3,1) 4	(3,2) 7	(3,3) X	(3,4) 13	(3,5) 16
r4	(4,1) 6	(4,2) X	(4,3) 17	(4,4) X	(4,5) 19
r5	(5,1) 9	(5,2) 12	(5,3) 15	(5,4) 18	(5,5) 20

Figure 4: BFS expansion order on the grid

d.7 Final Result

- **Path:** (1,1) -> (2,1) -> (3,1) -> (4,1) -> (5,1) -> (5,2) -> (5,3) -> (5,4) -> (5,5)
- **Actions:** Down, Down, Down, Down, Right, Right, Right, Right
- **Total cost:** 8 (equal to the Manhattan lower bound, so the path is optimal)
- **Nodes expanded:** 20 (every reachable free cell)

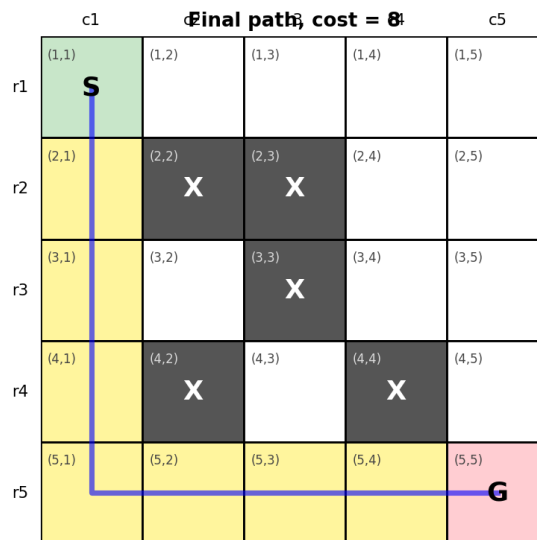


Figure 5: Final path found by BFS (cost = 8)

Interpretation: the robot goes down the left aisle and then along the bottom aisle to the goal. Because the goal is at the deepest level (depth 8), BFS had to expand all 20 reachable cells first, including the dead ends (3,2) and (4,3).

Part (e) - Depth-First Search

e.1 Algorithm and Assumptions

DFS uses a LIFO stack as the frontier and always expands the deepest node. Neighbours are explored in the order Up, Down, Left, Right, so they are pushed in reverse order (Right first, Up last) and Up is popped first. An explored set prevents loops. The goal test is applied when a node is popped.

e.2 Pseudocode

```
DFS(problem):
    stack <- [(START, None)]
    explored <- {}
    while stack is not empty:
        (node, par) <- stack.pop()
        if node in explored: continue
        explored.add(node); parent[node] <- par
        if Goal(node): return path(node)
        for each child in reversed(successors(node)):
            if child not in explored: stack.push((child, node))
    return failure
```

e.3 Flowchart

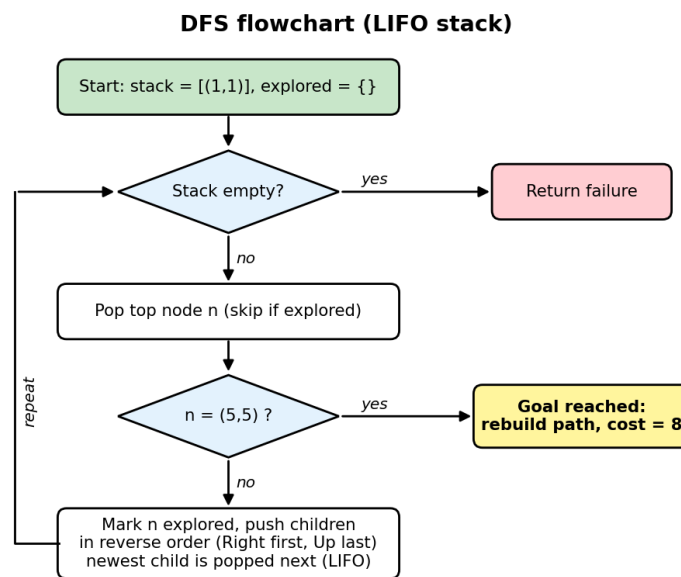


Figure 6: DFS flowchart

e.4 Python Code

```
start = (1, 1)
goal = (5, 5)

blocked = {
    (2, 2), (2, 3),
    (3, 3),
    (4, 2), (4, 4)
}

moves = [
    (-1, 0), # Up
```

```

    (1, 0), # Down
    (0, -1), # Left
    (0, 1) # Right
]

stack = [start]
visited = {start}
parent = {start: None}

while stack:
    current = stack.pop()

    if current == goal:
        break

    r, c = current

    for dr, dc in reversed(moves):
        next_cell = (r + dr, c + dc)

        if (1 <= next_cell[0] <= 5 and
            1 <= next_cell[1] <= 5 and
            next_cell not in blocked and
            next_cell not in visited):

            visited.add(next_cell)
            parent[next_cell] = current
            stack.append(next_cell)

path = []
current = goal

while current is not None:
    path.append(current)
    current = parent[current]

path.reverse()

print("DFS Path:")
print(path)
print("Total Cost:", len(path) - 1)

```

Output :

```
IDLE Shell 3.13.11
File Edit Shell Debug Options Window Help
Python 3.13.11 (tags/v3.13.11:6278944, Dec 5 2025, 16:26:58) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Nadesa Krishnan/OneDrive/Documents/DFS.py =====
DFS Path:
[(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
Total Cost: 8
>>>
```

e.5 Step-by-Step Expansion (Stack at Each Step)

The stack is written from bottom to top; the rightmost entry is popped next.

Step	Node popped / expanded	Children pushed	Stack after the step (bottom -> top)
1	(1,1)	(2,1), (1,2)	(1,2), (2,1)
2	(2,1)	(3,1)	(1,2), (3,1)
3	(3,1)	(4,1), (3,2)	(1,2), (3,2), (4,1)
4	(4,1)	(5,1)	(1,2), (3,2), (5,1)
5	(5,1)	(5,2)	(1,2), (3,2), (5,2)
6	(5,2)	(5,3)	(1,2), (3,2), (5,3)
7	(5,3)	(4,3), (5,4)	(1,2), (3,2), (5,4), (4,3)
8	(4,3)	none (dead end, backtrack)	(1,2), (3,2), (5,4)
9	(5,4)	(5,5)	(1,2), (3,2), (5,5)
10	(5,5)	GOAL REACHED	(1,2), (3,2) left unexplored

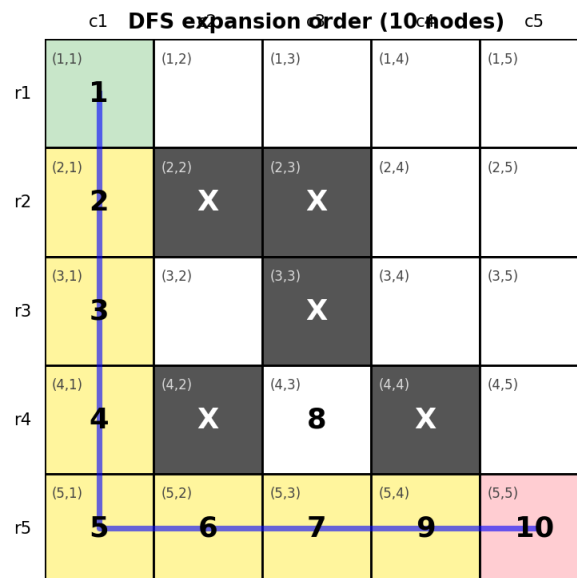


Figure 7: DFS expansion order and final path

e.6 Final Result

- **Path:** (1,1) -> (2,1) -> (3,1) -> (4,1) -> (5,1) -> (5,2) -> (5,3) -> (5,4) -> (5,5)
- **Total cost:** 8
- **Nodes expanded:** 10 (out of 20 free cells)
- **Backtracking:** one backtrack, at the dead end (4,3).

Interpretation: with the order Up, Down, Left, Right, DFS dives down the left column, runs along the bottom row, backtracks once from (4,3) and reaches the goal after only 10 expansions. It found a cost-8 path here, but this is a consequence of the chosen neighbour order: DFS does not guarantee the shortest path. In another layout or order it could return a longer route.

Part (f) - Comparison of BFS and DFS

Let b = branching factor (at most 4 here, about 2-3 in practice), d = depth of the shallowest goal ($d = 8$) and m = maximum depth of the search tree (bounded by the 20 states in graph search).

f.1 Comparison Table

Criterion	BFS	DFS	Application to this problem
Completeness	Complete (finite b , finite grid)	Complete only with repeated-state checking on a finite grid	Both are complete here: a visited/explored set is used and the grid has only 20 states.
Optimality	Optimal for uniform step cost	Not optimal in general	BFS guarantees cost 8; DFS reached cost 8 because of the neighbour order.
Time complexity	$O(b^d)$	$O(b^m)$	BFS expanded 20 nodes; DFS expanded 10 (its worst case can also reach 20).
Space complexity	$O(b^d)$: whole frontier stored	$O(b \cdot m)$: current path and siblings	Memory is not a concern on a 20-state grid (peak queue and stack size were both 4).
Data structure	FIFO queue	LIFO stack	-

f.2 Measured Results

Metric	BFS	DFS
Path cost	8	8
Nodes expanded	20	10
Dead ends visited	(3,2), (4,3)	(4,3)
Guaranteed shortest	Yes	No

f.3 Which Algorithm is More Suitable?

BFS is more suitable for this warehouse delivery problem.

1. The robot's performance measure is the shortest path and every move costs 1. BFS is guaranteed to return the minimum-cost path; DFS gives no such guarantee.
2. The state space is tiny (20 states), so BFS's higher memory use and extra expansions are negligible; DFS's memory advantage does not matter.
3. Warehouse layouts change (shelves added or moved). BFS stays correct for any layout, whereas DFS's result depends on the neighbour order and could produce a long detour that wastes battery and time.
4. DFS is attractive only when memory is very limited, the grid is huge, or any solution (not the best) is acceptable. None of these applies here.

Conclusion

The warehouse delivery problem can be modeled as a goal-based intelligent agent operating in a discrete grid environment. The robot starts at (1,1) and must reach (5,5) while avoiding blocked cells.

The problem can be formulated as a search problem using an initial state, goal state, actions, transition model, and path-cost function. Both BFS and DFS can find a solution in this finite warehouse environment.

For the given movement ordering, both algorithms produce an 8-movement path. However, BFS is the more appropriate algorithm because every movement has equal cost and BFS guarantees a shortest path. DFS does not guarantee optimality and may return a longer route depending on the order in which nodes are explored.